

PRACTICAL NO 5

Aim: Write a program to implement Stack ADT.

INPUT:

```
class Stack:
    def __init__(self):
        self.stack = []
    def push(self, item):
        self.stack.append(item)
    def pop(self):
        if len(self.stack) == 0:
            return None
        return self.stack.pop()
    def peek(self):
        return self.stack[-1]
    def isEmpty(self):
        return len(self.stack) == 0
    def display(self):
        print(self.stack)

print("----- Stack Operations -----")
st = Stack()

st.push(10)
st.push(20)
st.push(40)

print("Stack: ")
st.display()

print("Peek:", st.peek())
st.push(35)
print("Pop:", st.pop())

print("After Pop:")
st.display()
print()
```

```
print("----- Delimiter Matching -----")
```

```
s = "{[()]"
st = Stack()
for ch in s:
    if ch in "([{":
        st.push(ch)
    else:
        if st.isEmpty():
            print("Not Balanced")
            break
        top = st.pop()
        if ch == ')' and top != '(':
            print("Not Balanced")
            break
        if ch == '}' and top != '{':
            print("Not Balanced")
            break
        if ch == ']' and top != '[':
            print("Not Balanced")
            break
    else:
        if st.isEmpty():
            print("Balanced")
        else:
            print("Not Balanced")
```

```
print("----- Prefix to Postfix -----")
```

```
def isOperator(ch):
    return ch in ['+', '-', '*', '/']
def preToPost(preExp):
    st = Stack()
    preExp = preExp[::-1]
    print(preExp)
    for ch in preExp:
        if isOperator(ch):
            op1 = st.pop()
            op2 = st.pop()
            st.push(op1 + op2 + ch)
        else:
            st.push(ch)
    return st.pop()
ex = "*+-ABC*/DFG"
print("PreFix Expression: ", ex)
print("PostFix Expression: ", preToPost(ex))
```

OUTPUT:

```
PS D:\Coding\college-pr\DSA_PY> C:\Python34\python.  
----- Stack Operations -----  
Stack:  
[10, 20, 40]  
Peek: 40  
Pop: 35  
After Pop:  
[10, 20, 40]  
  
----- Delimiter Matching -----  
Balanced  
----- Prefix to Postfix -----  
PreFix Expression:  *+-ABC*/DFG  
GFD/*CBA-+*  
PostFix Expression:  AB-C+DF/G**  
PS D:\Coding\college-pr\DSA_PY> █
```